System for Artificial Intelligence

# Hardware Acceleration

Siyuan Feng
Shanghai Innovation Institute

# Recap: Overview of Machine Learning Systems

上海创智学院
Shanghai Innovation Institute



## ML Models

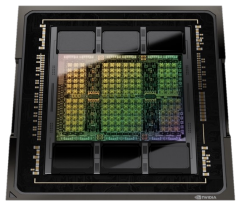
Automatic Differentiation

Graph Optimization

Parallelism / Distributed

Hardware Acceleration

This Lecture



NVIDIA GPU



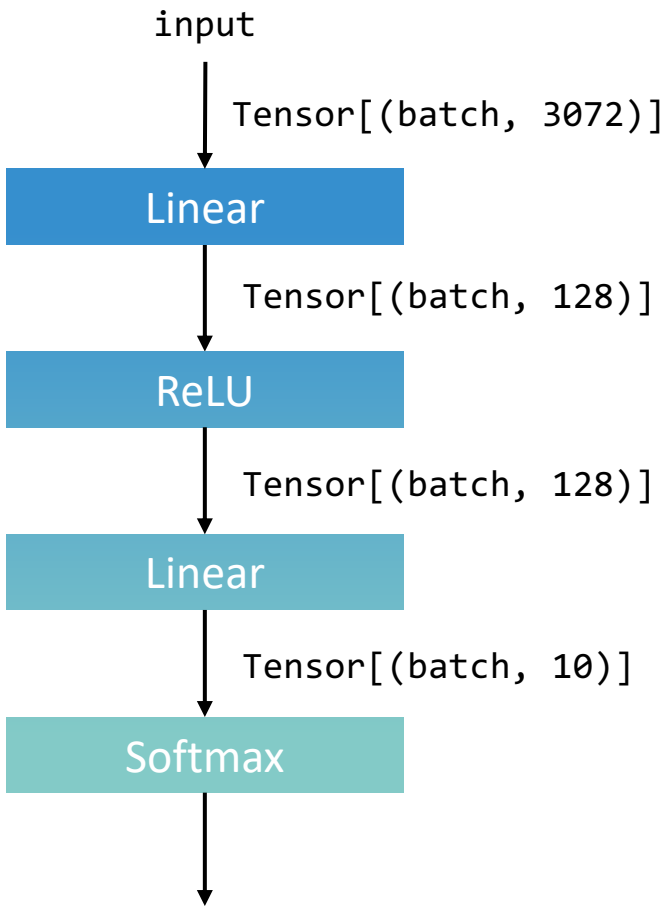

HUAWEI NPU



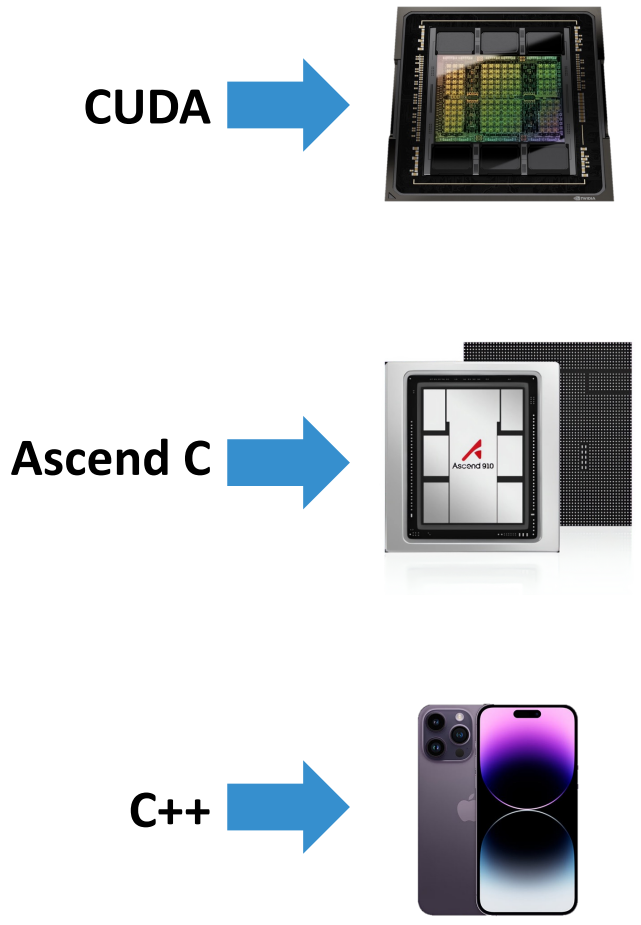
Mobile devices

# Discussion: Building a Minimal ML Runtime

上海创智学院
Shanghai Innovation Institute

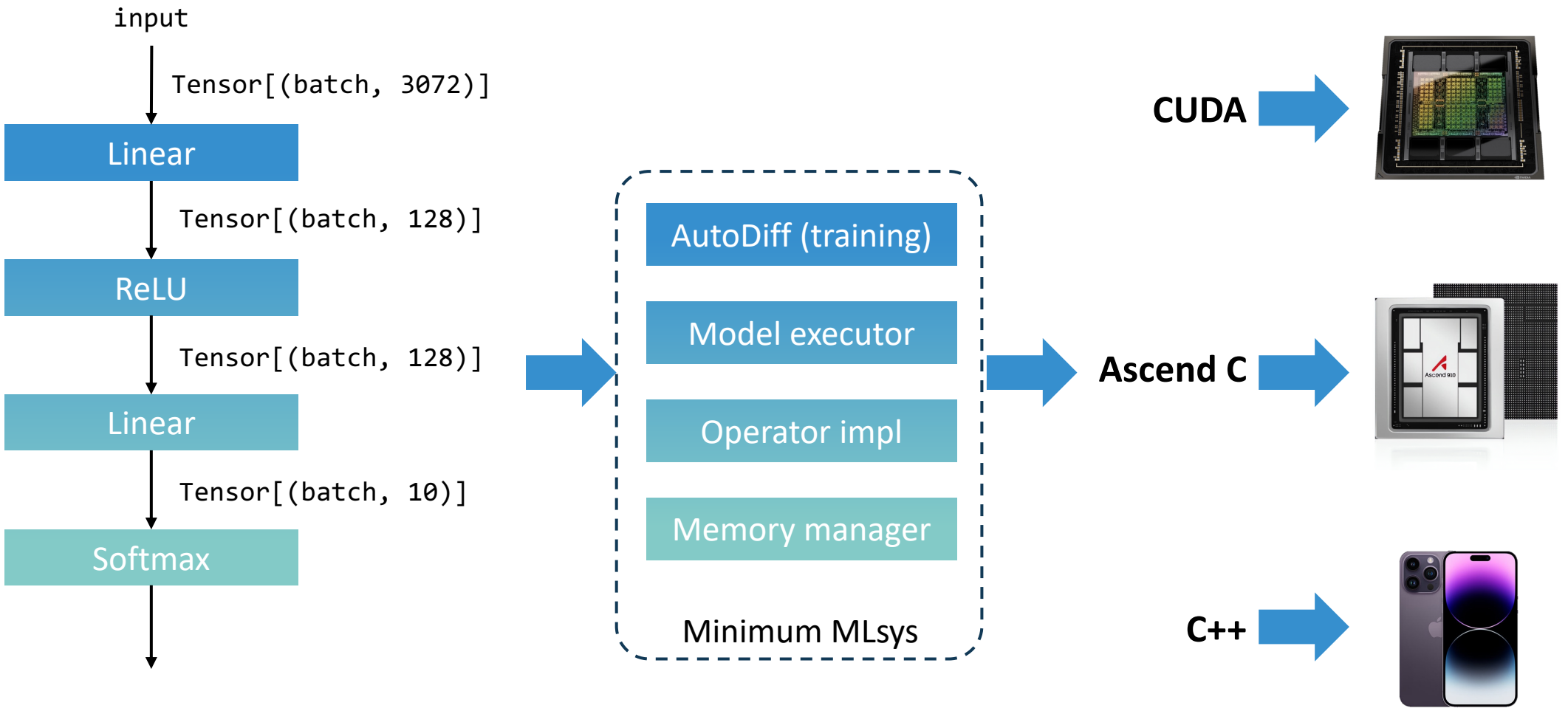



?



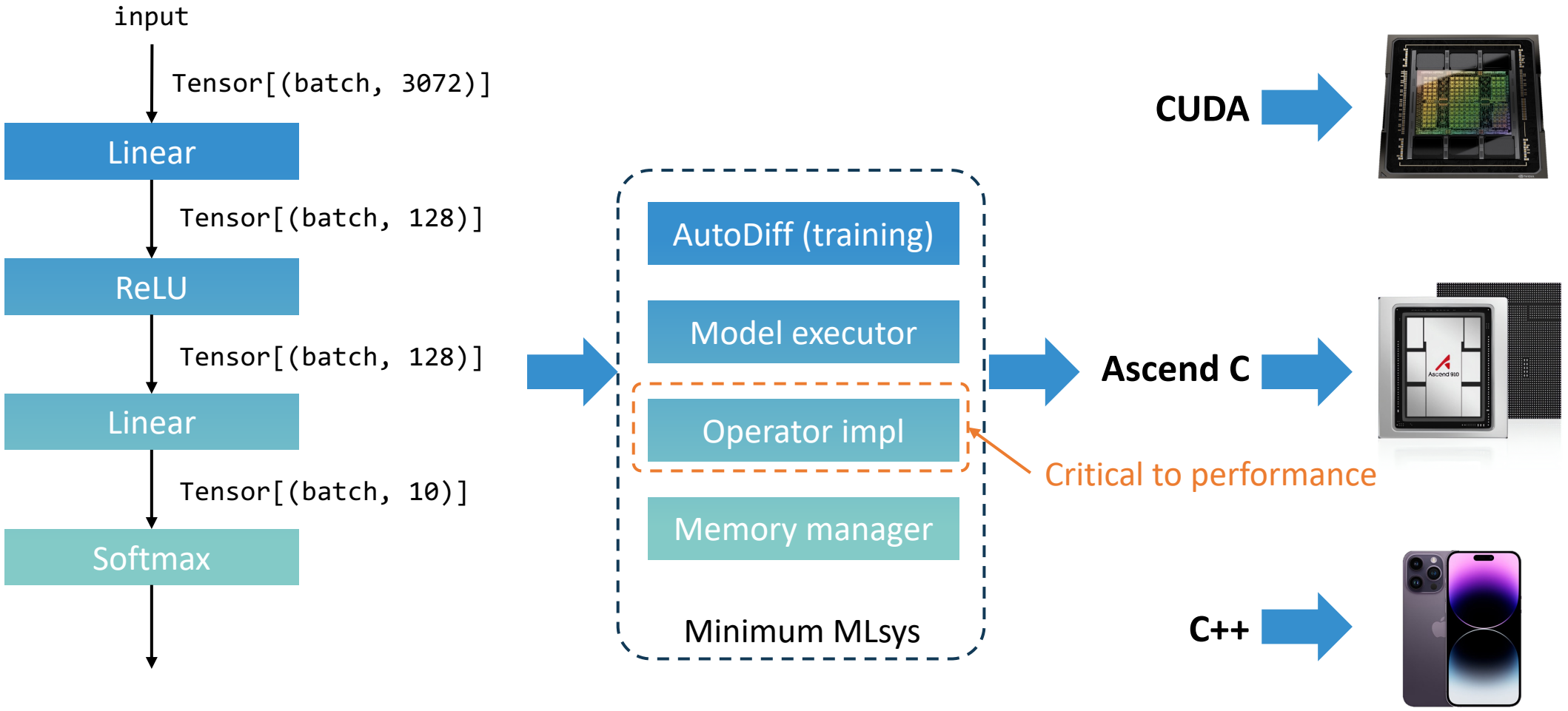
# Discussion: Building a Minimal ML Runtime

上海创智学院
Shanghai Innovation Institute



# Discussion: Building a Minimal ML Runtime

上海创智学院
Shanghai Innovation Institute



# OUTLINE

01 General acceleration techniques

Parallel work, data access, and resource limits

02 Case study: matrix multiplication

Reuse, blocking, and end-to-end measurement

01

# General acceleration techniques

# Vectorization

上海创智学院
Shanghai Innovation Institute

Pseudocode: 256 FP32 elements per array, 4 bytes per element

```
void vec_add(float* A, float *B, float* C) {
  for (int i = 0; i < 64; ++i) {
    float4 a = load_float4(A + i*4);
    float4 b = load_float4(B + i*4);
    float4 c = add_float4(a, b);
    store_float4(C + i*4, c);
  }
}
```

This example assumes 16-byte-aligned loads and stores.

Alignment requirements depend on the instruction.

# Parallelization

上海创智学院
Shanghai Innovation Institute

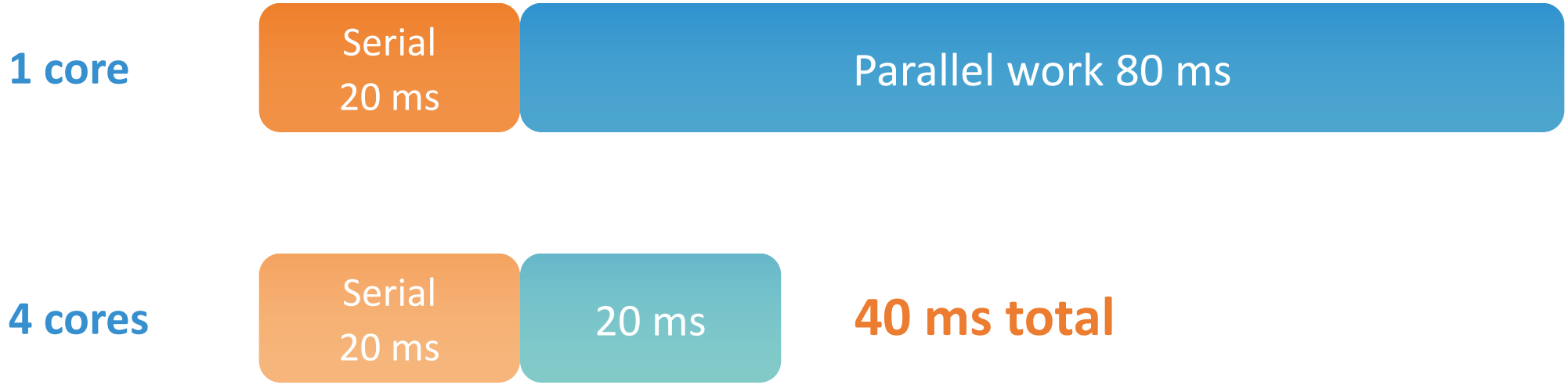
```
void parallel_vec_add(float* A, float *B, float* C) {
  #pragma omp parallel for
  for (int i = 0; i < 64; ++i) {
    float4 a = load_float4(A + i*4);
    float4 b = load_float4(B + i*4);
    float4 c = add_float4(a, b);
    store_float4(C + i*4, c);
  }
}
```

Independent iterations can run on multiple threads.

Small arrays may not amortize thread scheduling overhead.

# Parallel Speedup and the Serial Fraction






$$Speedup = \frac{100}{40} = 2.5 \quad Limit = \frac{100}{20} = 5$$

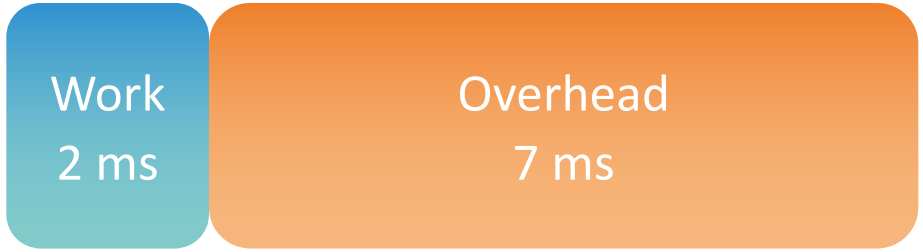
Which part should we optimize next?

Fixed workload. Ideal load balance. No thread, synchronization, or bandwidth overhead.

# Task Granularity and Load Balance

上海创智学院
Shanghai Innovation Institute

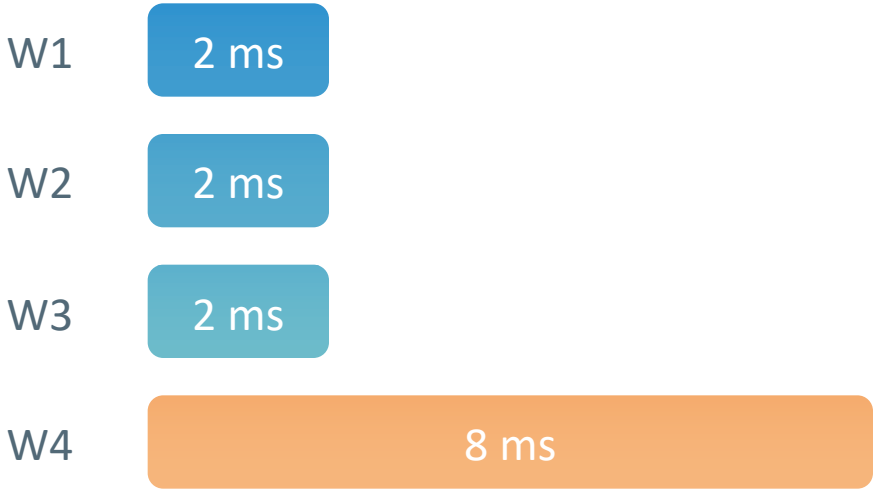
## Small tasks




4 workers: 9 ms

Serial baseline: 8 ms

## Uneven tasks



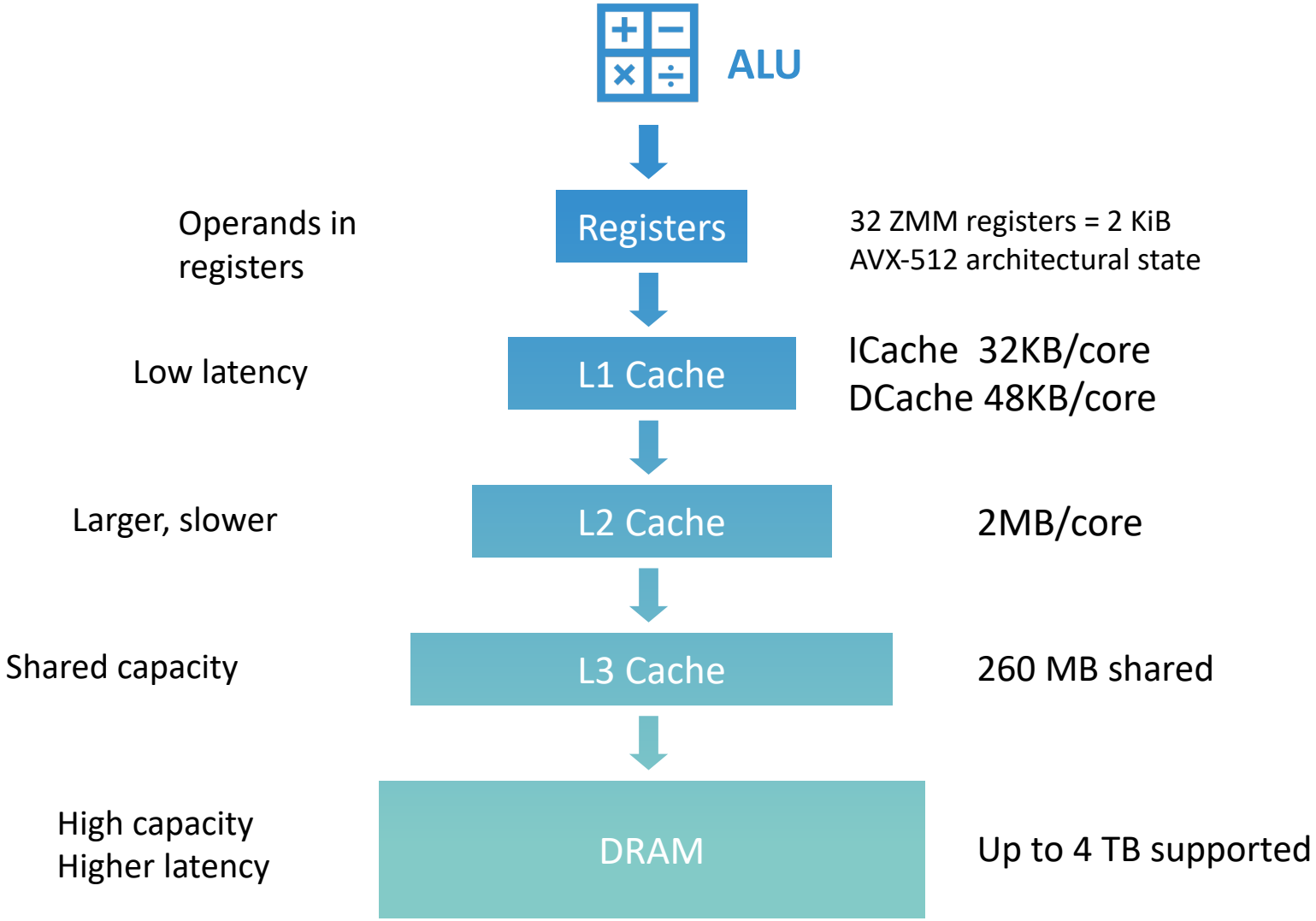

Finish time: 8 ms

After balancing work, can memory keep the cores busy?

Teaching examples. Timings include the stated overhead only. Workers start together.

# CPU Memory Hierarchy: Xeon 8558 Example






Intel Xeon Platinum 8558 (2023); latency depends on access conditions.

# Row-Major Layout: Indices and Addresses

上海创智学院
Shanghai Innovation Institute

## Logical A[3][4]

| | | | |
|---|---|---|---|
| 0 | 1 | 2 | 3 |
| 4 | 5 | 6 | 7 |
| 8 | 9 | 10 | 11 |

## A[i][j] byte offset

$$(4i + j) \times 4$$

Next column: +4 bytes

Next row: +16 bytes

## Physical order

| | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |

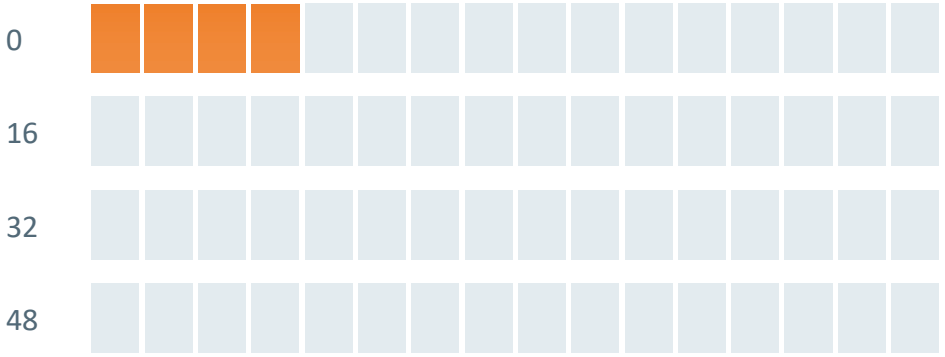
A 3 × 4 FP32 matrix in row-major order. This small diagram explains addresses, not timings.

# Data Layout and Cache-Line Use

上海创智学院
Shanghai Innovation Institute

## Contiguous access

`A[i], i = 0...3`




**4 loads touch 1 cache line**

## Strided access

`A[16*i], i = 0...3`




**4 loads touch 4 cache lines**

How does loop order change the stride?

FP32, aligned 64-byte cache lines, cold cache. First 4 loads only. No prefetch.

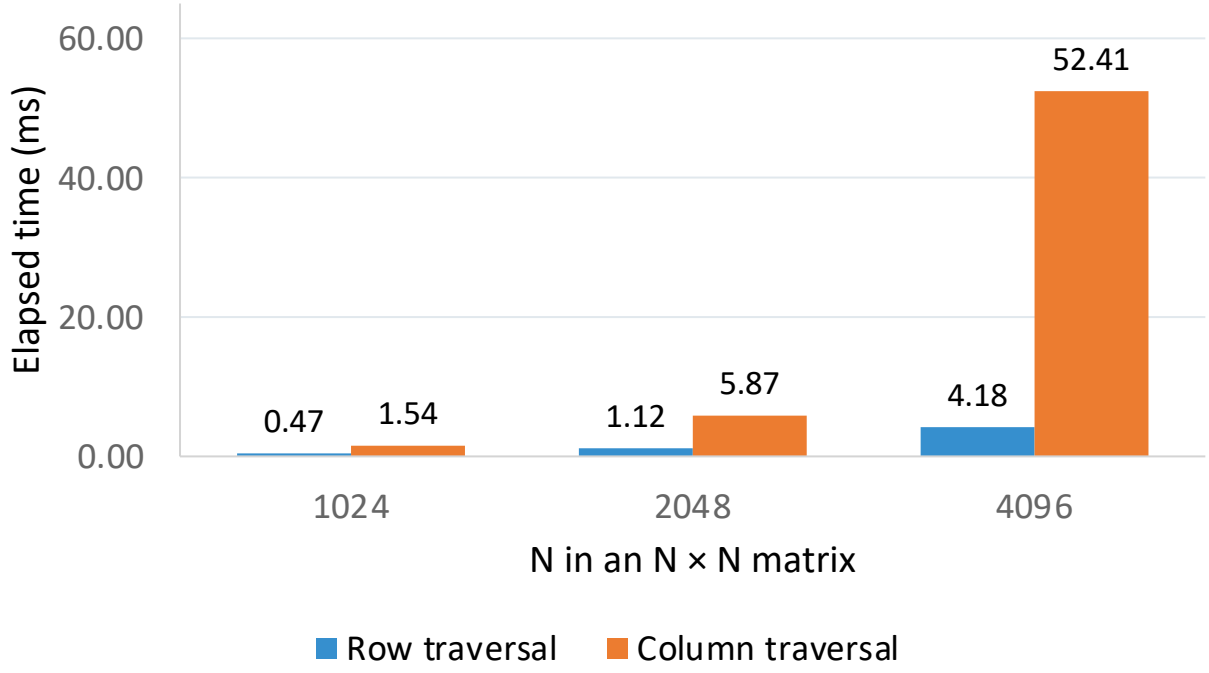
# CPU Experiment: Row and Column Traversal

上海创智学院
Shanghai Innovation Institute

## Same data and exact sum

```
Row traversal
for i in range(N):
  for j in range(N):
    s += A[i*N + j]

Column traversal
for j in range(N):
  for i in range(N):
    s += A[i*N + j]
```




M4 Max, 1 thread, uint32, SIMD off. 11 runs after warm-up; no cache eviction. 2026-09-18.

# NumPy: A Transpose View and a Contiguous Copy

上海创智学院
Shanghai Innovation Institute

```
a = np.arange(12, dtype=np.float32).reshape(3, 4)
t = a.T
p = np.ascontiguousarray(t)
```

| Property | a | t = a.T | p |
|---|---|---|---|
| Shape | (3, 4) | (4, 3) | (4, 3) |
| Strides (bytes) | (16, 4) | (4, 16) | (12, 4) |
| Shares memory with a | Yes | Yes | No |

t and p have identical values, but different physical layouts.

Executed NumPy example. Strides are in bytes. Transposing a view changes metadata only.

# Views, Packing, and Reuse

上海创智学院
Shanghai Innovation Institute

## Same backing storage

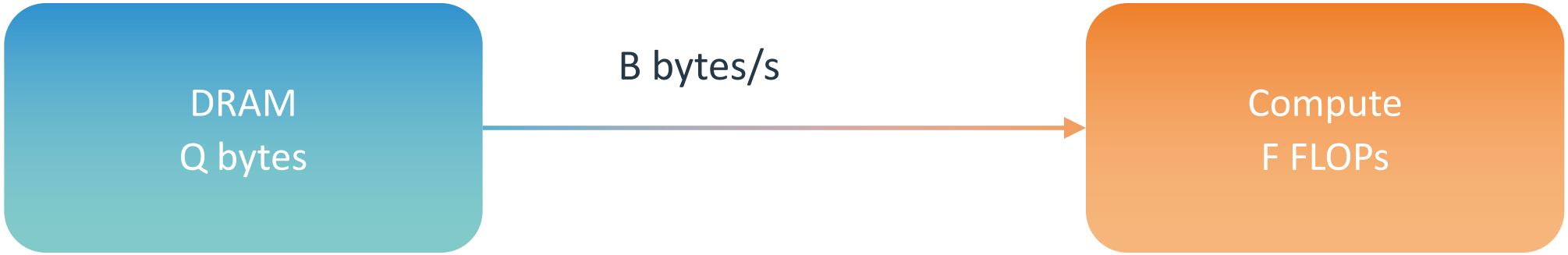



$$Baseline\colon 2K\ ms \quad Packed\colon (4 + K)\ ms$$

How many reuses pay for the 4 ms packing cost?

Teaching example: same input reused K times. Kernel 2 ms → 1 ms, packing once 4 ms.

# Compute and Data Movement




P FLOP/s

$$T \geq max(\frac{F}{P}, \frac{Q}{B})$$

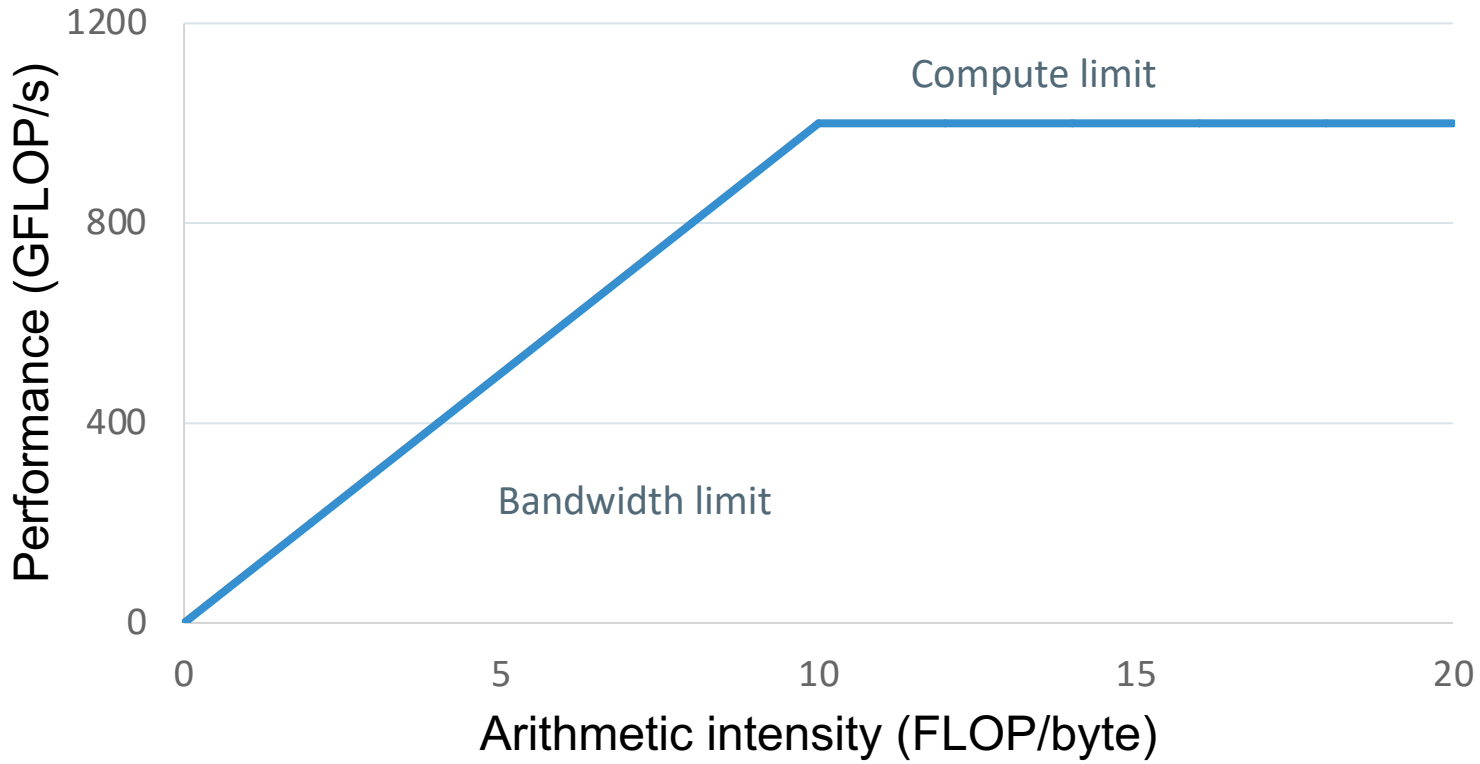
$$I = \frac{F}{Q}$$

How do parallelism and data reuse affect these limits?

F: work  Q: DRAM traffic  I: arithmetic intensity (FLOP/byte)

# The Roofline Model

$$Performance \leq min(P, B \times I)$$




**Teaching hardware**

1,000 GFLOP/s

100 GB/s

Ridge point

10 FLOP/byte

Linear axes. The roof is an upper bound, not a promised performance.

# Vector Addition: Counting Work and Bytes

上海创智学院
Shanghai Innovation Institute

```
for i in range(N):  C[i] = A[i] + B[i]
```




$$F = N \quad Q = 12N \quad I = \frac{1}{12} \approx 0.083$$

Why can more SIMD lanes still hit a bandwidth limit?

FP32. Ideal DRAM traffic: two input reads and one output write per element. No write allocation.

# Data Reuse and the Roofline Bound

上海创智学院
Shanghai Innovation Institute

| Implementation A | Implementation B |
| --- | --- |
| Work: 20 GFLOP | Work: 20 GFLOP |
| DRAM traffic: 10 GB | DRAM traffic: 1 GB |
| Intensity: 2 FLOP/byte | Intensity: 20 FLOP/byte |
| **Roof: 200 GFLOP/s** | **Roof: 1,000 GFLOP/s** |
| Time ≥ 100 ms | Time ≥ 20 ms |

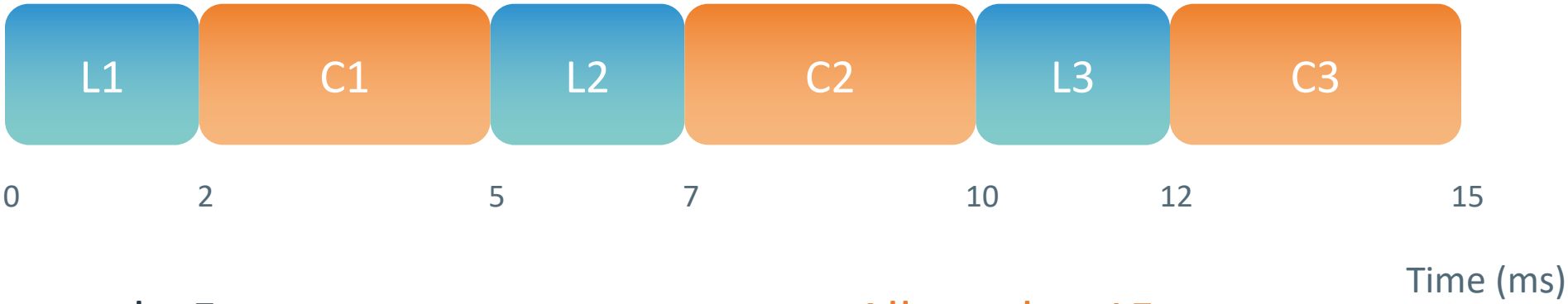
With fewer bytes to move, can load and compute also overlap?

Teaching assumptions: 1,000 GFLOP/s, 100 GB/s. Bounds exclude other overheads.

# Pipelining: A Serial Schedule

3 chunks  Load: 2 ms/chunk  Compute: 3 ms/chunk

## One buffer




First result: 5 ms

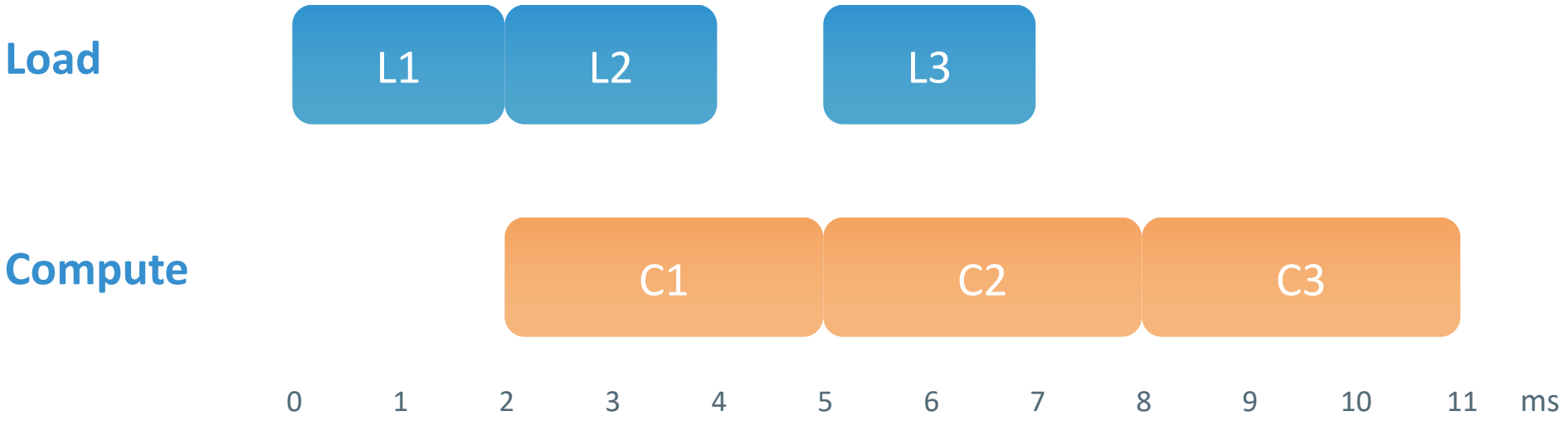
All results: 15 ms

Can the next load run while this chunk computes?

Teaching assumptions. No overlap, launch overhead, or output transfer.

# Pipelining: Overlapping Load and Compute

Independent load and compute resources



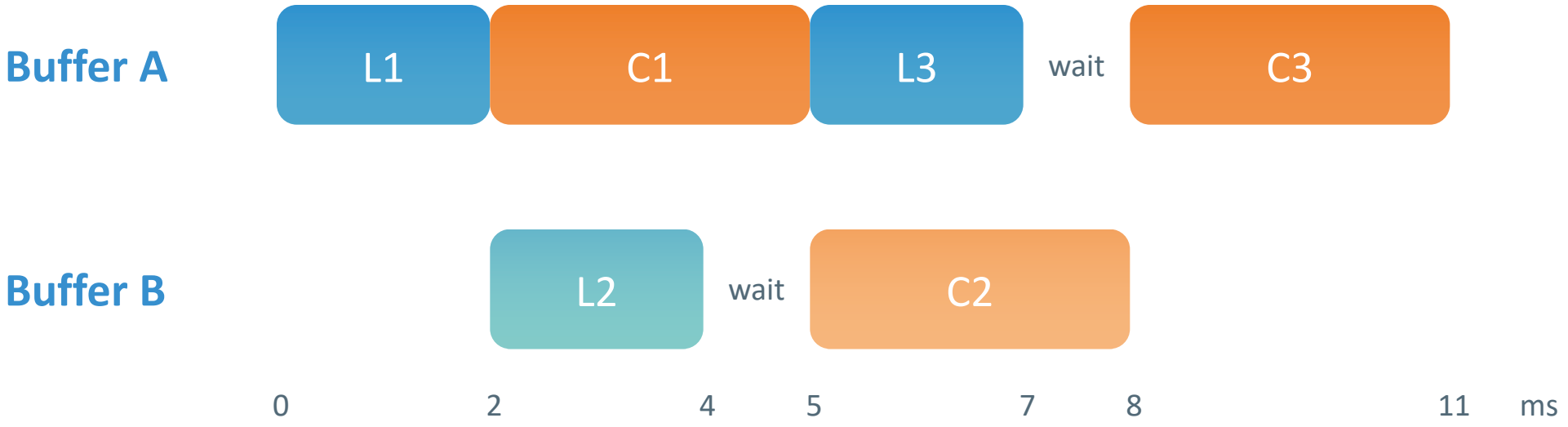

First result: 5 ms

All results: 11 ms

Steady state: one result every 3 ms. Actual overlap requires hardware and runtime support.

# Double Buffering: Safe Buffer Reuse

Load writes a buffer. Compute reads the same buffer.




Why must L3 wait until C1 releases buffer A?

Wait for load completion before reading. Wait for compute completion before overwriting.

02

# Case study: matrix multiplication

How can a linear layer reuse its inputs?

# Linear Layer Workload: Batch and Weights

上海创智学院
Shanghai Innovation Institute

```
Y = X @ W.T
```

FP32, no bias or activation

W: 4096 × 4096
64 MiB

## Batch 1

X: 1 × 4096

## Batch 64



X: 64 × 4096

Which inputs can a matrix multiplication reuse?

Ideal reuse: load the weights once per batch. Actual DRAM traffic depends on the implementation.

# Vanilla Matrix Multiplication

上海创智学院
Shanghai Innovation Institute

Compute `C = dot(A, B.T)`

```
float A[n][n], B[n][n], C[n][n];

for (int i = 0; i < n; ++i) {
  for (int j = 0; j < n; ++j) {
    C[i][j] = 0;
    for (int k = 0; k < n; ++k) {
      C[i][j] += A[i][k] * B[j][k];
    }
  }
}
```

$\Theta(N^3)$

Row-major inputs; B stores one output feature per row.

# Matrix Multiplication: Reuse Opportunities

上海创智学院
Shanghai Innovation Institute

```
float A[n][n];
float B[n][n];
float C[n][n];


C[i][j] = sum(A[i][k] * B[j][k], axis=k)
```

Access of $A$ is independent of $j$,
tile the $j$ dimension by $t$ enables reuse of $A$ for $t$ times.

Reuse B across input rows to share weights within a batch.

# Architecture Aware Analysis

上海创智学院
Shanghai Innovation Institute

Baseline: no input reuse across outputs; caches ignored.

```
dram float A[n][n], B[n][n], C[n][n];

for (int i = 0; i < n; ++i) {
  for (int j = 0; j < n; ++j) {
    register float c = 0;
    for (int k = 0; k < n; ++k) {
      register float a = A[i][k];
      register float b = B[j][k];
      c += a * b;
    }
    C[i][j] = c;
  }
}
```

A reads (elements): $n^3$
B reads (elements): $n^3$

Only A/B reads are counted; per-element costs are idealized.

Read-cost model: $2 * C_{dram \to reg} * n^3$

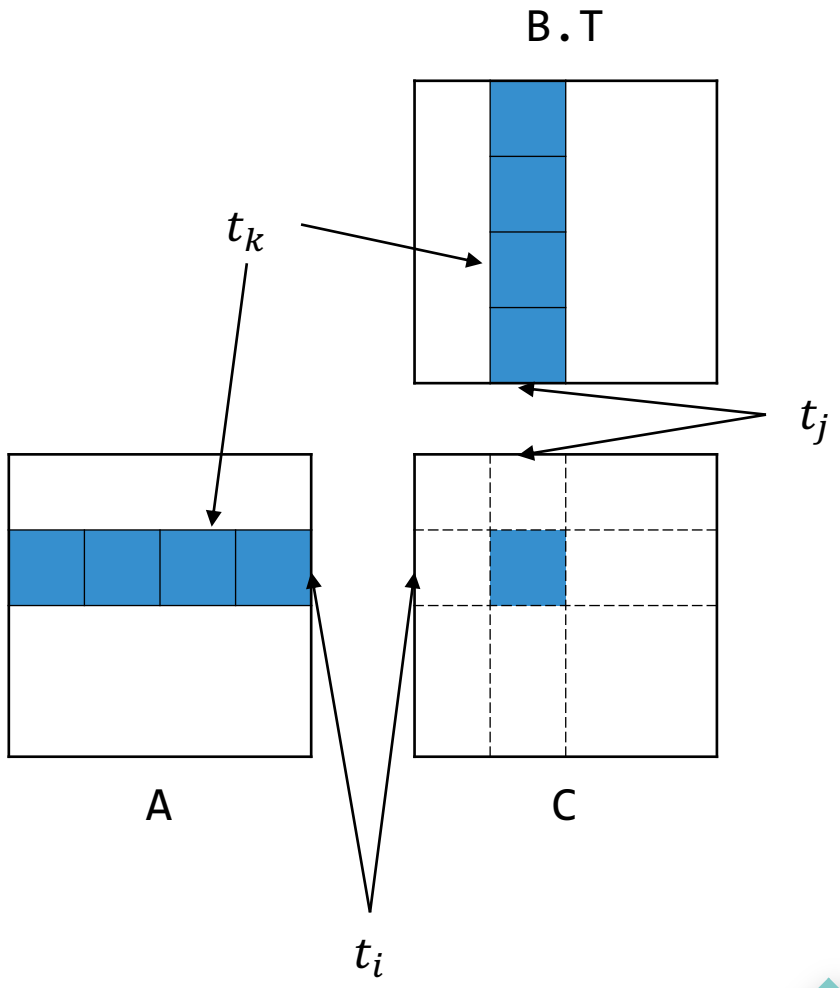
# Register Tiled Matrix Multiplication

上海创智学院
Shanghai Innovation Institute

Two-level model; ignore caches. Packed inputs; tile sizes divide n.

```
dram float A[n/ti][n/tk][ti][tk];
dram float B[n/tj][n/tk][tj][tk];
dram float C[n/ti][n/tj][ti][tj];

for (int i = 0; i < n/ti; ++i) {
  for (int j = 0; j < n/tj; ++j) {
    register float c[ti][tj] = 0;
    for (int k = 0; k < n/tk; ++k) {
      register float a[ti][tk] = A[i][k];
      register float b[tj][tk] = B[j][k];
      c += dot(a, b.T);
    }
    C[i][j] = c;
  }
}
```



# Register Tiled Matrix Multiplication

上海创智学院 Shanghai Innovation Institute

Two-level model; ignore caches. Packed inputs; tile sizes divide n.

```
dram float A[n/ti][n/tk][ti][tk];
dram float B[n/tj][n/tk][tj][tk];
dram float C[n/ti][n/tj][ti][tj];

for (int i = 0; i < n/ti; ++i) {
  for (int j = 0; j < n/tj; ++j) {
    register float c[ti][tj] = 0;
    for (int k = 0; k < n/tk; ++k) {
      register float a[ti][tk] = A[i][k];
      register float b[tj][tk] = B[j][k];
      c += dot(a, b.T);
    }
    C[i][j] = c;
  }
}
```

A reads (elements): $n^3/t_j$

B reads (elements): $n^3/t_i$

Read-cost model: $C_{dram \to reg} \left( \frac{n^3}{t_i} + \frac{n^3}{t_j} \right)$

Discussion: Why choose $t_k$

# Register Tiled Matrix Multiplication

上海创智学院
Shanghai Innovation Institute

Two-level model; ignore caches. Packed inputs; tile sizes divide n.

```
dram float A[n/ti][n/tk][ti][tk];
dram float B[n/tj][n/tk][tj][tk];
dram float C[n/ti][n/tj][ti][tj];

for (int i = 0; i < n/ti; ++i) {
  for (int j = 0; j < n/tj; ++j) {
    register float c[ti][tj] = 0;
    for (int k = 0; k < n/tk; ++k) {
      register float a[ti][tk] = A[i][k];
      register float b[tj][tk] = B[j][k];
      c += dot(a, b.T);
    }
    C[i][j] = c;
  }
}
```

A storage (elements): $t_i \times t_k$
B storage (elements): $t_j \times t_k$
C storage (elements): $t_i \times t_j$

FP32 bytes = 4 × elements. Actual register allocation may spill.

Read-cost model: $C_{dram \to reg} \left( \frac{n^3}{t_i} + \frac{n^3}{t_j} \right)$

Total elements: $t_i \times t_k + t_j \times t_k + t_i \times t_j$

# Cache Blocking

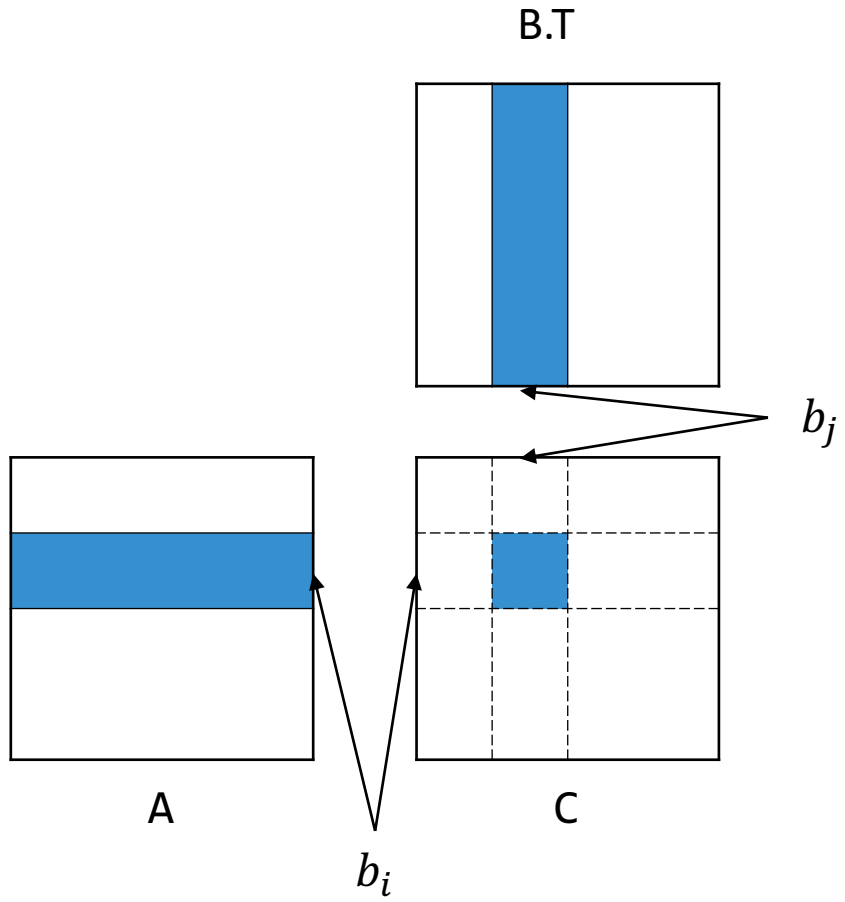
Idealized cache residency; packed inputs; block sizes divide n.

```
dram float A[n/bi][bi][n];
dram float B[n/bj][bj][n];
dram float C[n/bi][n/bj][bi][bj];

for (int i = 0; i < n/bi; ++i) {
  l1_cache float a[bi][n] = A[i];
  for (int j = 0; j < n/bj; ++j) {
    l1_cache float b[bj][n] = B[j];

    C[i][j] = dot(a, b.T);
  }
}
```

Sub-procedure, can apply register tiling here




Question: Cache capacity and A/B reads (DRAM to L1)

# Cache Blocking

Idealized cache residency; packed inputs; block sizes divide n.

```
dram float A[n/bi][bi][n];
dram float B[n/bj][bj][n];
dram float C[n/bi][n/bj][bi][bj];

for (int i = 0; i < n/bi; ++i) {
  l1_cache float a[bi][n] = A[i];
  for (int j = 0; j < n/bj; ++j) {
    l1_cache float b[bj][n] = B[j];

    C[i][j] = dot(a, b.T);
  }
}
```

A reads into L1 (elements): $n^2$
B reads into L1 (elements): $n^3/b_i$

A cache storage (elements): $b_i \times n$
B cache storage (elements): $b_j \times n$

A/B panels and other live data must fit in usable cache.
FP32 bytes = 4 × elements.

# Combining Register and Cache Reuse

上海创智学院
Shanghai Innovation Institute

```
dram float A[n/bi][bi/ti][n][ti];
dram float B[n/bj][bj/tj][n][tj], C[n][n];
for (int i = 0; i < n/bi; ++i) {
  l1_cache float a[bi/ti][n][ti] = A[i];
  for (int j = 0; j < n/bj; ++j) {
    l1_cache float b[bj/tj][n][tj] = B[j];
    for (int x = 0; x < bi/ti; ++x)
    for (int y = 0; y < bj/tj; ++y) {
      register float c[ti][tj] = 0;
      for (int k = 0; k < n; ++k) {
        register float ar[ti] = a[x][k][:];
        register float br[tj] = b[y][k][:];
        c += outer(ar, br);
      }
      store_tile(C, i*bi+x*ti, j*bj+y*tj, c);
    }
  }
}
```

A/B read-cost model: $C_{dram\to l1}\times\left(n^2+\frac{n^3}{b_i}\right)+C_{l1\to reg}\times\left(\frac{n^3}{t_i}+\frac{n^3}{t_j}\right)$

Pseudocode; idealized cache residency. Excludes packing, C traffic and overlap.

# Linear Layers: Ideal Reuse and Performance

上海创智学院
Shanghai Innovation Institute

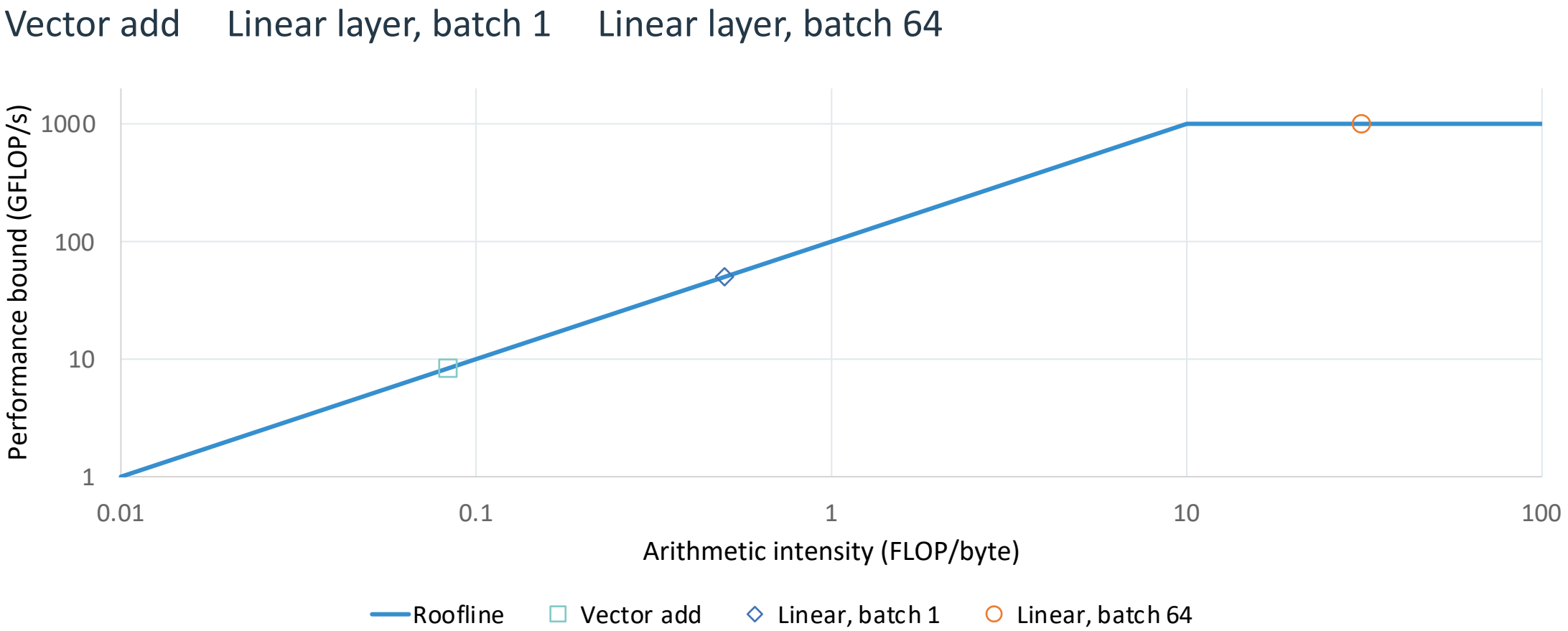
Same 4096 × 4096 FP32 weights; batch 1 versus 64

| Quantity | Batch 1 | Batch 64 |
|---|---|---|
| Work (GFLOP) | 0.0336 | 2.1475 |
| Ideal DRAM traffic (MiB) | 64.03125 | 66 |
| Intensity (FLOP/byte) | 0.50 | 31.03 |
| Performance bound (GFLOP/s) | 49.98 | 1,000 |
| Batch time lower bound (ms) | 0.6714 | 2.1475 |

Higher throughput does not imply a shorter batch latency.

Ideal traffic, not a traffic count of the preceding kernel. 1,000 GFLOP/s, 100 GB/s.
FMA = 2 FLOPs. 1 MiB = $2^{20}$ bytes.

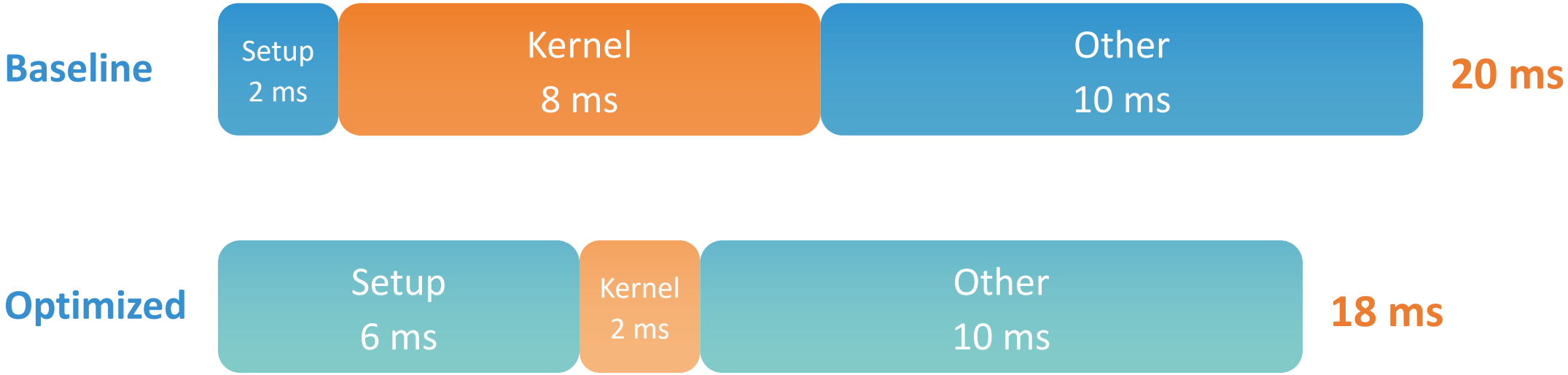
# Roofline: Vector Addition and Linear Layers




Logarithmic axes. Markers show model bounds, not measurements. Same teaching hardware.

# Kernel Speed and End-to-End Speed






**Kernel: 4×**

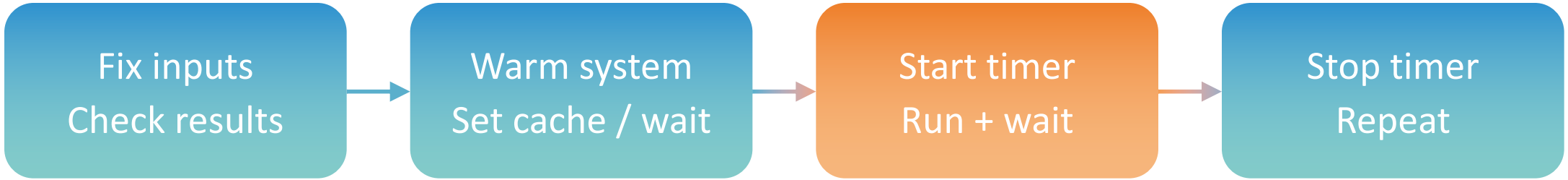
**End-to-end: 1.11×**

What if setup grows to 10 ms?

One request, serial stages. Same output and precision. Teaching timings, not measured results.

# A Reproducible Measurement

上海创智学院
Shanghai Innovation Institute




## Teaching samples (ms)

| 2.0 | 2.1 | 2.0 | 5.8 | 2.2 |
|---|---|---|---|---|

**Median: 2.1 ms**

Did the timer include completed work or only submission?

Report timing scope, cache policy, workload, hardware, threads, repetitions, and variation.

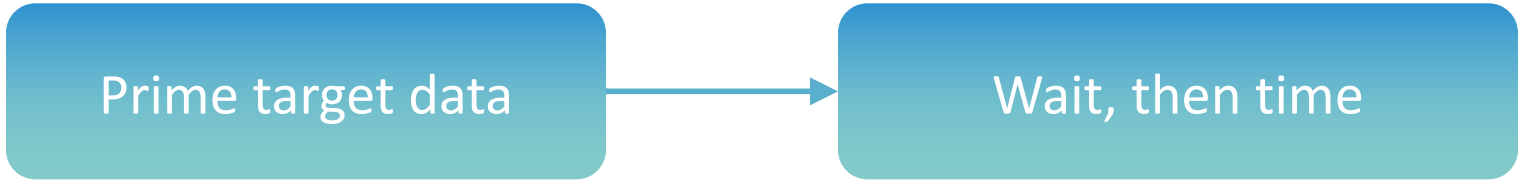
# Measurement: Cache State Matters

上海创智学院
Shanghai Innovation Institute

Warmup: stabilize clocks and finish one-time setup.

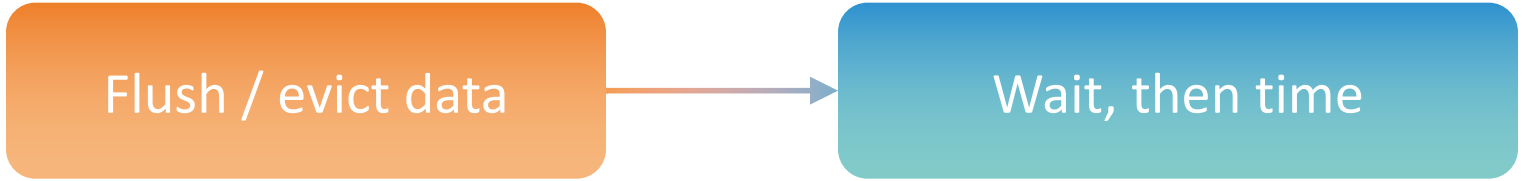
Then prepare the data cache before each timed sample.

**Repeated-input test**




Preparation outside timing

**Cold-cache test**




Preparation outside timing

Application test: preserve the real producer / consumer history.

Which cache state matches the workload you want to measure?

Repeat cache preparation before every sample. Synchronization does not flush data caches.

# Cache Preparation: A CPU Experiment

上海创智学院
Shanghai Innovation Institute

Same uint32 sum; only the pre-run memory access changes.

## Median of 40 samples per condition

| Input | Repeated input | After sweep | Ratio |
|---|---|---|---|
| 1 MiB | 9.0 µs | 12.0 µs | 1.33× |
| 4 MiB | 37.0 µs | 46.0 µs | 1.24× |
| 64 MiB | 695.5 µs | 749.0 µs | 1.08× |

Sweep: read a separate 256 MiB buffer before each timed run.

A different starting state changes the measured time.

M4 Max CPU, 1 thread, clang -O3. Sweep excluded. Approximate eviction; no miss counters.

# Possible Reuse Pattern in Convolution

上海创智学院
Shanghai Innovation Institute

Stride = 1, no padding, dilation = 1; h and w are at least K.

```
float Input[n][ci][h][w];
float Weight[co][ci][K][K];
float Output[n][co][h-K+1][w-K+1];

Output[b][o][y][x] =
    sum(Input[b][k][y+ry][x+rx] * Weight[o][k][ry][rx],
        axis=[k, ry, rx]);
```

$0 \le b < n$; $0 \le o < co$; $0 \le y < h-K+1$; $0 \le x < w-K+1$
Sum over $0 \le k < ci$ and $0 \le ry, rx < K$.

Which output dimensions can reuse an input or weight?

# Optional: Strassen's Algorithm

Optional extension: reduce operation count. This slide uses C = AB.

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}, C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \cdot \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

Thus, we have

$$\begin{aligned} C_{11} &= A_{11} \cdot B_{11} + A_{12} \cdot B_{21} \\ C_{12} &= A_{11} \cdot B_{12} + A_{12} \cdot B_{22} \\ C_{21} &= A_{21} \cdot B_{11} + A_{22} \cdot B_{21} \\ C_{22} &= A_{21} \cdot B_{12} + A_{22} \cdot B_{22} \end{aligned}$$

$$\begin{aligned} M_1 &= (A_{11} + A_{22}) \cdot (B_{11} + B_{22}) \\ M_2 &= (A_{21} + A_{22}) \cdot B_{11} \\ M_3 &= A_{11} \cdot (B_{12} - B_{22}) \\ M_4 &= A_{22} \cdot (B_{21} - B_{11}) \\ M_5 &= (A_{11} + A_{12}) \cdot B_{22} \\ M_6 &= (A_{21} - A_{11}) \cdot (B_{11} + B_{12}) \\ M_7 &= (A_{12} - A_{22}) \cdot (B_{21} + B_{22}) \end{aligned}$$

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} M_1 + M_4 - M_5 + M_7 & M_3 + M_5 \\ M_2 + M_4 & M_1 - M_2 + M_3 + M_6 \end{bmatrix}$$

**8 multiplications in normal algorithm**

**7 multiplications in Strassen algorithm**

Fewer multiplications come with extra additions and temporary storage.

**The general complexity is $\Theta\left(n^{\log_2 7}\right)$**

# Acknowledgement

The development of this course, including its structure, content, and accompanying presentation slides, has been significantly influenced and inspired by the excellent work of instructors and institutions who have shared their materials openly. We wish to extend our sincere acknowledgement and gratitude to the following courses, which served as invaluable references and a source of pedagogical inspiration:

- Machine Learning Systems[15-442/15-642], by **Tianqi Chen** and **Zhihao Jia** at **CMU**.
- Advanced Topics in Machine Learning (Systems)[CS6216], by **Yao Lu** at **NUS**

While these materials provided a foundational blueprint and a wealth of insightful examples, all content herein has been adapted, modified, and curated to meet the specific learning objectives of our curriculum. Any errors, omissions, or shortcomings found in these course materials are entirely our own responsibility. We are profoundly grateful for the contributions of the educators listed above, whose dedication to teaching and knowledge-sharing has made the creation of this course possible.

System for Artificial Intelligence

# Thanks

Siyuan Feng
Shanghai Innovation Institute